ДИПЛОМНЫЙ ПРОЕКТ

на тему:

«WILDMARKET»

Сайт для покупок онлайн

(аналог маркетплейса Wildberries)

Курс:

Разработчик веб-приложений на Python

Нестеров Илья Ярославович

2025-2026

СОДЕРЖАНИЕ

I. Введение и описание проекта

1.1. О проекте и цели разработки

1.2. Функциональные требования

II. Технологический стэк

2.1. Серверная часть (Backend)

2.2. Клиентская часть (Frontend)

2.3. Инфраструктура

III. Структура базы данных и архитектура

3.1. Модели данных

3.2. Структура проекта

IV. Реализация ключевых модулей

4.1. Аутентификация и смена ролей

4.2. Каталог и страница товара

4.3. Корзина и оформление заказа

4.4. Кабинет продавца

4.5. Кабинет администратора

V. Развертывание и тестирование

5.1. CI/CD и облачное развертывание

5.2. Тестирование

Заключение

I. ВВЕДЕНИЕ И ОПИСАНИЕ ПРОЕКТА

1.1. О проекте и цели разработки

Целью дипломной работы является разработка полнофункционального интернет-магазина «WildMarket» — аналога российского маркетплейса Wildberries. Задача охватывает весь цикл создания веб-приложения: проектирование, разработку, развёртывание и тестирование.

1.2. Функциональные требования

Система реализует трёхролевую модель пользователей, соответствующую реальной бизнес-логике маркетплейса:

Покупатель (buyer)

— просмотр каталога из 8 категорий товаров с карточками, ценами, скидками и рейтингами;

— поиск товаров по названию, фильтрация по категории и цене;

— регистрация, авторизация, управление профилем;

— добавление товаров в избранное и корзину;

— оформление заказа с указанием адреса доставки;

— просмотр истории заказов и их текущего статуса;

— написание отзывов и выставление оценок (1–5).

Продавец (seller)

— личный кабинет с аналитикой: всего товаров, активные, на модерации, выручка;

— добавление новых товаров: название, цена, описание, категория, остаток, фото;

— редактирование и удаление собственных товаров.

Администратор (admin)

— сводная панель: статистика по пользователям, товарам, заказам и выручке;

— модерация товаров — подтверждение или отклонение товаров от продавцов;

— управление категориями (создание, удаление);

— просмотр всех пользователей и заказов, смена статуса заказа.

II. ТЕХНОЛОГИЧЕСКИЙ СТЕК

Архитектура приложения построена по шаблону MVC (Model-View-Controller): Flask обрабатывает HTTP-запросы и бизнес-логику, Jinja2 формирует HTML-ответ, SQLAlchemy взаимодействует с PostgreSQL. Приложение запускается через WSGI-сервер Gunicorn на облачной платформе Render.

2.1. Серверная часть (Backend)

— Python 3.11 — основной язык разработки.

— Flask — микрофреймворк: маршруты, представления, обработка форм, управление сессиями.

— SQLAlchemy (Flask-SQLAlchemy) — ORM для работы с базой данных; 6 моделей с миграциями через Flask-Migrate.

— Flask-Login — управление сессиями пользователей, защита маршрутов декоратором @login_required.

— Flask-WTF / WTForms — генерация и валидация форм (регистрация, добавление товара).

— Werkzeug — хэширование паролей: generate_password_hash / check_password_hash.

— Gunicorn — WSGI-сервер для production-запуска приложения.

2.2. Клиентская часть (Frontend)

— Jinja2 — шаблонизатор с наследованием: базовый шаблон base.html содержит общие шапку и футер.

— Bootstrap 5.3 — адаптивная сетка, компоненты (карточки, навигация, формы, модальные окна).

— HTML5 / CSS3 — разметка и кастомные стили.

— JavaScript (Vanilla) — динамическое обновление корзины и избранного без перезагрузки страницы.

2.3. Инфраструктура

— PostgreSQL — основная реляционная база данных; экземпляр развёрнут на Render.

— GitHub — система контроля версий; push в ветку main запускает автоматический деплой.

— Render.com — облачная платформа: хостинг приложения и базы данных 24/7.

III. СТРУКТУРА БАЗЫ ДАННЫХ И АРХИТЕКТУРА

3.1. Модели данных

База данных содержит 6 моделей SQLAlchemy, описывающих всю предметную область маркетплейса:

— User — пользователь системы: поля username, email, password_hash, role (buyer / seller / admin).

— Product — товар: name, price, old_price, description, image, stock, category, seller_id (FK → User), is_approved.

— Order — заказ: user_id (FK → User), total, status (Новый / Подтверждён / Доставлен / Отменён), address, created_at.

— OrderItem — позиция в заказе: order_id (FK → Order), product_id (FK → Product), quantity, price.

— CartItem — элемент корзины: user_id, product_id, quantity.

— Favorite — избранное: user_id, product_id.

Связи: Product.seller_id → User; Order → [OrderItem] → Product; CartItem и Favorite образуют связь многие-ко-многим между User и Product.

3.2. Структура проекта

Кодовая база организована по принципу Blueprint-архитектуры Flask:

```
app/
  routes/       — маршруты по модулям: auth, catalog, cart, seller, admin
  models.py     — ORM-модели всех таблиц
  templates/    — Jinja2-шаблоны с наследованием от base.html
  static/       — CSS, JavaScript, изображения
config.py       — конфигурация: DATABASE_URL, SECRET_KEY
requirements.txt— зависимости проекта
start.sh        — скрипт запуска: миграции + Gunicorn
```

IV. РЕАЛИЗАЦИЯ КЛЮЧЕВЫХ МОДУЛЕЙ

4.1. Аутентификация и система ролей

Вход в систему и управление сессиями реализованы через Flask-Login. Состояние аутентификации хранится в подписанной cookie-сессии. Пароли не сохраняются в открытом виде — применяется

функция generate_password_hash (Werkzeug), при проверке используется check_password_hash.

Каждый пользователь имеет поле role. Доступ к защищённым маршрутам контролируется декораторами:

— @login_required — закрывает корзину, заказы и кабинеты от неаутентифицированных пользователей.

— @admin_required — проверяет role == 'admin', при несоответствии возвращает HTTP 403.

— @seller_required — аналогично для продавцов.

Реализована регистрация через Google OAuth 2.0: пользователь может войти на сайт, используя свой аккаунт Google, без создания отдельного пароля;

В личном кабинете добавлена возможность смены пароля: пользователь вводит текущий пароль, новый пароль и его подтверждение, после чего хеш пароля обновляется в базе данных.

4.2. Каталог и страница товара

Каталог реализован как список карточек с поддержкой поиска (LIKE-запрос по полю name) и фильтрации по категории. Отображаются 8 категорий. На каждой карточке выводятся: изображение, название, цена, старая цена (если есть скидка), рейтинг (среднее отзывов).

Страница товара содержит: полное описание, наличие на складе, имя продавца, список отзывов с оценками и форму для добавления нового отзыва (только для авторизованных покупателей).

Покупатель может оставить отзыв только после получения заказа — это гарантирует, что оценка и комментарий основаны на реальном опыте использования товара. На странице товара отображается средний рейтинг (рассчитывается автоматически на основе всех оценок) и список отзывов с указанием имени пользователя, даты и текста. При добавлении нового отзыва рейтинг товара пересчитывается в реальном времени.

4.3. Корзина и оформление заказа

Добавление товара в корзину и избранное выполняется через AJAX-запросы без перезагрузки страницы. Корзина отображает список позиций, их количество и итоговую сумму. При оформлении заказа создаётся запись Order и связанные OrderItem; остаток товара (stock) уменьшается.

При оформлении заказа покупатель выбирает способ доставки: курьерская доставка по адресу, самовывоз из ПВЗ СДЭК или отделения Почты России. Доступные варианты доставки и их стоимость автоматически подгружаются через API, а пункты выдачи отображаются на Яндекс.Карте для наглядного выбора.

4.4. Кабинет продавца

Продавец видит сводку своих товаров: всего, активные (is_approved=True), на модерации (is_approved=False), суммарную выручку по завершённым заказам. Все товары продавца доступны для редактирования и удаления. Новый товар попадает в статус «На модерации» до одобрения администратором.

4.5. Панель администратора

Панель предоставляет глобальную статистику и инструменты управления: список всех пользователей с возможностью просмотра, очередь товаров на модерацию (одобрить / отклонить), управление категориями, список всех заказов со сменой статуса.

V. РАЗВЁРТЫВАНИЕ И ТЕСТИРОВАНИЕ

5.1. CI/CD и облачное развёртывание

Для публикации сайта используется платформа Render.com. Схема непрерывного деплоя (CI/CD):

— Исходный код проекта хранится в репозитории GitHub (github.com/NesterPlay12/diplom_project).

— Render подключён к репозиторию: любой push в ветку main запускает автоматическую пересборку.

— Скрипт start.sh выполняет миграции базы данных (flask db upgrade) и запускает Gunicorn.

— База данных PostgreSQL развёрнута на серверах Render (регион Frankfurt), подключение через переменную окружения DATABASE_URL.

Конфигурационный файл config.py автоматически нормализует строку подключения (postgres:// → postgresql://) для совместимости с SQLAlchemy и предусматривает fallback на SQLite при локальной разработке.

5.2. Тестирование

После каждого значимого изменения проводилось ручное функциональное тестирование на развёрнутом сайте. Для проверки всех ролей созданы тестовые учётные записи:

Администратор: presentation.admin@example.invalid | 1XZ05EueBVIMcyxhWuCqlUzC!Aa7

Модератор: presentation.moderator@example.invalid | -vX1ogSYMiuzR5BSyVnRPePZ!Aa7

Продавец: presentation.seller@example.invalid | GryL60PRV-gz4UIYs9BWnhEN!Aa7

Покупатель: presentation.user@example.invalid | Jy1fot552xPYaVMOjA9gYI_i!Aa7

Проверены следующие сценарии: регистрация и вход, добавление товара продавцом, прохождение модерации, поиск и фильтрация в каталоге, добавление в корзину/избранное, оформление заказа, смена статуса заказа администратором (Новый → Подтверждён → Доставлен), написание отзыва. Также вход через Google OAuth и смена пароля в личном кабинете, выбор доставки (СДЭК / Почта России) и отображение пунктов выдачи на карте.

ЗАКЛЮЧЕНИЕ

В результате выполнения дипломной работы был разработан, развёрнут и протестирован полнофункциональный интернет-магазин «WildMarket» — аналог маркетплейса Wildberries, работающий в режиме 24/7.

Все поставленные задачи выполнены в полном объёме:

— реализована трёхролевая система: покупатель, продавец, администратор;

— разработан полный цикл торговых операций: каталог → карточка товара → корзина → заказ;

— реализованы вспомогательные функции: избранное, отзывы с рейтингом, поиск и фильтрация;

— настроены личные кабинеты для всех ролей, включая панель аналитики продавца;

— выполнено развёртывание на Render с базой данных PostgreSQL и автоматическим CI/CD через GitHub;

— проведено ручное тестирование всех ключевых сценариев работы системы.

Проект демонстрирует применение современного стека веб-разработки (Python, Flask, SQLAlchemy, PostgreSQL, Bootstrap) для создания сложного многопользовательского приложения с разграничением ролей и полноценной бизнес-логикой маркетплейса.

Сайт проекта: https://wildmarket.su

Код: https://github.com/NesterPlay12/diplom_project